Integración doble y múltiple

Reglas del rectángulo y del trapecio, y cuadratura de Gauss–Legendre

Whady Felipe Flórez

Métodos Numéricos – Pregrado

Objetivos y ruta de la clase (60 min)

Al terminar la clase, el estudiante podrá:

1. Extender una regla de integración 1D a 2D y 3D mediante el producto tensorial.
2. Aplicar las reglas del rectángulo, del trapecio y de Gauss–Legendre en dominios rectangulares y no rectangulares.
3. Comparar los métodos por error y por número de evaluaciones de la función.
4. Reconocer cuándo una cuadratura funciona mal y cómo corregirlo.

Bloque	min
Motivación y paso de 1D a 2D	10
Rectángulos y trapecios en 2D	13
Cuadratura de Gauss–Legendre	17
Dominios no rectangulares	7
Integrales triples y dimensión	5
Python, síntesis y ejercicios	8
Total	**60**

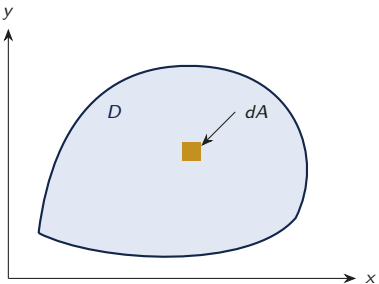
Motivación: integrales dobles en ingeniería mecánica

Sobre una región plana D (placa, sección transversal, compuerta):

$$\text{Masa:} \quad m = \iint_D \rho(x,y)\,dA, \qquad \bar{x} = \frac{1}{m}\iint_D x\,\rho\,dA$$

$$\text{Momento polar:} \quad J = \iint_D (x^2 + y^2)\,dA$$

$$\text{Fuerza de presión:} \quad F = \iint_D p(x,y)\,dA$$



$$\iint_D f\,dA \approx \sum_k w_k\, f(x_k, y_k)$$

En la práctica ρ, p o la forma de D impiden una primitiva cerrada, o solo se conocen valores en puntos (sensores, malla de simulación).

Idea clave. Toda cuadratura numérica es una suma ponderada de valores de la función. Los métodos difieren solo en *dónde* se evalúa f y con *qué peso*.

De una a dos dimensiones

Repaso 1D: rectángulo (punto medio) y trapecio

Con n subintervalos, $h = (b-a)/n$:

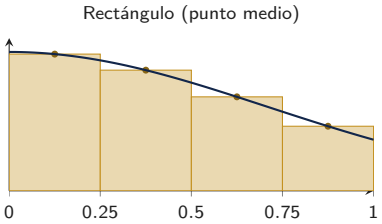
$$\int_a^b f\,dx \approx M_n = h\sum_{i=1}^{n} f\left(\tfrac{x_{i-1}+x_i}{2}\right)$$

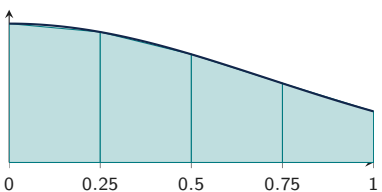
$$\int_a^b f\,dx \approx T_n = \frac{h}{2}\Big[f_0 + 2\sum_{i=1}^{n-1} f_i + f_n\Big]$$

Errores (compuestos):

$$I - M_n \approx +\frac{(b-a)h^2}{24}\bar{f}'', \qquad I - T_n \approx -\frac{(b-a)h^2}{12}\bar{f}''$$

Ambos son de orden $O(h^2)$, y el error del rectángulo es la mitad y de signo contrario al del trapecio.





La idea central: integral iterada $\Rightarrow$ producto tensorial

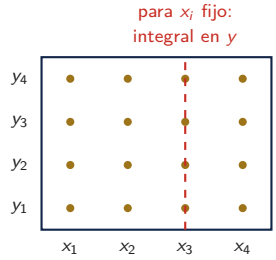
En un rectángulo $[a,b]\times[c,d]$ (teorema de Fubini):

$$I=\int_a^b \underbrace{\left(\int_c^d f(x,y)\,dy\right)}_{g(x)}\,dx$$

Aplicamos una regla 1D con pesos w_j a la integral interior y otra regla 1D con pesos w_i a la exterior:

$$g(x)\approx\sum_j w_j\,f(x,y_j) \quad\Longrightarrow\quad \boxed{I\approx\sum_i\sum_j w_i\,w_j\,f(x_i,y_j)}$$

Los nodos 2D forman una malla $\{x_i\}\times\{y_j\}$ y el peso de cada nodo es el producto de los pesos 1D.



Idea clave. No hay que aprender métodos nuevos: cualquier regla 1D se convierte en una regla 2D (o 3D) multiplicando pesos.

Rectángulos y trapecios en 2D

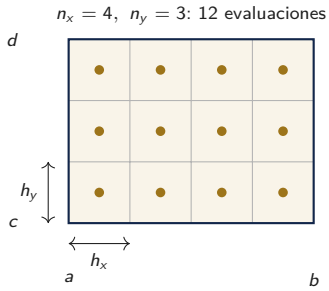
Regla del rectángulo (punto medio) en 2D

Dividimos $[a,b]\times[c,d]$ en $n_x \times n_y$ celdas de tamaño $h_x \times h_y$ y evaluamos f en el centro de cada celda:

$$I \approx M = h_x h_y \sum_{i=1}^{n_x}\sum_{j=1}^{n_y} f\left(\bar{x}_i, \bar{y}_j\right)$$

$$\bar{x}_i = a + \left(i - \tfrac{1}{2}\right)h_x, \qquad \bar{y}_j = c + \left(j - \tfrac{1}{2}\right)h_y$$

- Interpretación: suma de *prismas* de base $h_x h_y$ y altura $f(\bar{x}_i, \bar{y}_j)$.
- $n_x n_y$ evaluaciones; no usa la frontera (útil si f es singular en el borde).
- Error: $O(h_x^2 + h_y^2)$; exacta para funciones bilineales.



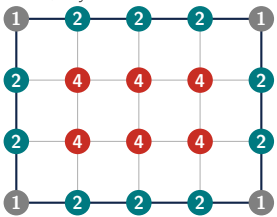
Regla del trapecio en 2D: la plantilla de pesos 1–2–4

Producto de dos trapecios 1D con pesos $(1, 2, 2, \ldots, 2, 1)$:

$$I \approx T = \frac{h_x h_y}{4} \sum_{i=0}^{n_x} \sum_{j=0}^{n_y} c_{ij}\, f(x_i, y_j), \qquad c_{ij} = c_i\, c_j$$

- Esquinas: $c_{ij} = 1$; bordes: $c_{ij} = 2$; interior: $c_{ij} = 4$.
- Cada nodo interior lo comparten 4 celdas: de ahí el 4.
- $(n_x + 1)(n_y + 1)$ evaluaciones; usa los valores de la frontera (útil con datos de una malla o de sensores).
- Error: $O(h_x^2 + h_y^2)$.

$n_x = 4$, $n_y = 3$: 20 evaluaciones



Pregunta para la clase

Con la misma malla, ¿cuál cree que da menor error, el rectángulo o el trapecio? ¿Por cuánto?

Ejemplo de referencia

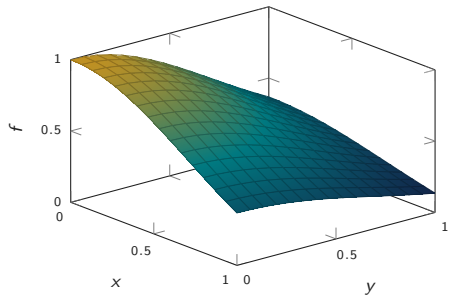
$$I = \int_0^1 \int_0^1 e^{-(x^2+y^2)}\, dy\, dx$$

No tiene primitiva elemental, pero sí un valor exacto porque es separable:

$$I = \left(\int_0^1 e^{-x^2} dx \right)^2 = \left(\frac{\sqrt{\pi}}{2} \operatorname{erf}(1) \right)^2$$

$$I = 0.557\,746\,285\,4$$

Este valor exacto permite medir el error real de cada método. (En un problema real no se conoce: se estima refinando la malla.)



Resultados: rectángulo vs. trapecio

n	Rectángulo M		Trapecio T		
	eval.	error	eval.	error	E_M/E_T
2	4	$+1.17 \times 10^{-2}$	9	-2.28×10^{-2}	-0.511
4	16	$+2.88 \times 10^{-3}$	25	-5.72×10^{-3}	-0.503
8	64	$+7.16 \times 10^{-4}$	81	-1.43×10^{-3}	-0.501
16	256	$+1.79 \times 10^{-4}$	289	-3.58×10^{-4}	-0.500
32	1024	$+4.47 \times 10^{-5}$	1089	-8.94×10^{-5}	-0.500

Lo que muestra la tabla:

- Al duplicar n ($h \to h/2$) el error cae ≈ 4 veces: orden 2.
- El rectángulo, con *menos* evaluaciones, tiene la *mitad* del error.
- Los errores tienen signos opuestos y razón $-\frac{1}{2}$, igual que en 1D.

Reto rápido

Si $E_M \approx -\frac{1}{2} E_T$, ¿qué combinación de M y T cancela el error de orden h^2?

Respuesta al reto: una regla de orden 4 “gratis”

Si $M = I + E$ y $T = I - 2E$, entonces

$$S = \frac{2M + T}{3} = I + O(h^4)$$

Es la misma idea que lleva de trapecio a Simpson en 1D (y la base de la extrapolación de Richardson).

n	$S = (2M + T)/3$	error
2	0.557 912 9	1.67×10^{-4}
4	0.557 756 6	1.03×10^{-5}
8	0.557 746 9	6.46×10^{-7}

Ahora el error cae ≈ 16 veces al duplicar n.

Idea clave. Conocer la estructura del error vale tanto como el método: con dos reglas de orden 2 se construye una de orden 4. Pero, ¿se puede llegar aún más lejos eligiendo mejor *dónde* evaluar? $\Rightarrow$ Gauss.

Cuadratura de Gauss–Legendre

Gauss–Legendre en 1D: nodos y pesos óptimos

$$\int_{-1}^{1} f(t)\,dt \approx \sum_{k=1}^{n} w_k\, f(t_k)$$

- $2n$ parámetros libres (n nodos y n pesos) $\Rightarrow$ la regla es exacta para polinomios de grado $\leq 2n-1$.
- Los nodos t_k son las raíces del polinomio de Legendre $P_n(t)$; los pesos son positivos.
- Rectángulo y trapecio de un intervalo son exactos solo hasta grado 1.
- Los nodos son *irracionales*: Gauss sirve cuando podemos evaluar f donde queramos (no con datos tabulados).

n	t_k	w_k	exacta
1	0	2	grado 1
2	$\pm 1/\sqrt{3}$ $= \pm 0.577\,350$	1	grado 3
3	0	$8/9$	grado 5
	$\pm\sqrt{3/5}$ $= \pm 0.774\,597$	$5/9$	



Cambio de variable: de $[-1,1]^2$ a $[a,b]\times[c,d]$

Transformación afín en cada dirección:

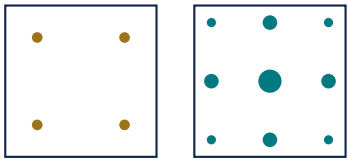
$$x = \frac{b-a}{2}\,t + \frac{a+b}{2}, \qquad y = \frac{d-c}{2}\,s + \frac{c+d}{2}$$

$$dx\,dy = |J|\,dt\,ds, \qquad |J| = \frac{(b-a)(d-c)}{4}$$

Regla de Gauss producto tensorial $n \times n$:

$$I \approx \frac{(b-a)(d-c)}{4} \sum_{i=1}^{n}\sum_{j=1}^{n} w_i\, w_j\, f\big(x(t_i),\, y(s_j)\big)$$

Exacta para f de grado $\leq 2n-1$ en cada variable por separado.



2×2 (4 nodos) $\quad$ 3×3 (9 nodos)

en 3×3 el tamaño del punto es proporcional a su peso $w_i w_j$

Pregunta para la clase

¿Qué error típico aparece si se olvida el jacobiano $(b-a)(d-c)/4$ en un dominio $[0,2]\times[0,1]$?

Ejemplo resuelto a mano: Gauss 2×2

Dominio $[0,1]^2$: $x = \frac{1}{2}t + \frac{1}{2}$, $|J| = \frac{1}{4}$, $w_i = w_j = 1$.

$$x_{1,2} = y_{1,2} = \tfrac{1}{2} \mp \tfrac{1}{2\sqrt{3}} = 0.211\,325,\ 0.788\,675$$

$$I \approx \frac{1}{4} \sum_{i=1}^{2} \sum_{j=1}^{2} f(x_i, y_j) = \frac{2.229\,615}{4}$$

$$I \approx \mathbf{0.557\,404} \qquad (\text{error } -3.4 \times 10^{-4})$$

x_i	y_j	$f(x_i, y_j)$
0.211 325	0.211 325	0.914 556
0.211 325	0.788 675	0.513 417
0.788 675	0.211 325	0.513 417
0.788 675	0.788 675	0.288 224
	Suma	2.229 615

Idea clave. Con solo 4 evaluaciones, Gauss 2×2 logra el mismo error que el trapecio con $n = 16$, que usa 289. Con las mismas 4 evaluaciones, el rectángulo ($n = 2$) tiene error 1.2×10^{-2}, unas 34 veces mayor.

Exactitud polinómica: un ejemplo que Gauss resuelve sin error

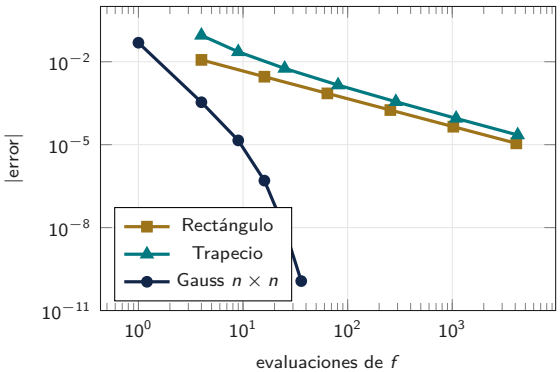
$$I = \int_0^2 \int_0^1 \left(x^3 y^2 + xy\right) dy\, dx = \frac{7}{3} = 2.333\,333$$

Grado 3 en x y grado 2 en $y \Rightarrow$ Gauss 2×2 (exacta hasta grado 3 en cada variable) debe dar el valor exacto.

Método	eval.	resultado
Gauss 1×1	1	1.500 000
Gauss 2×2	**4**	**2.333 333** (exacto)
Rectángulo $n = 8$	64	2.317 749
Trapecio $n = 8$	81	2.364 746

Idea clave. Gauss no "adivina": integra exactamente el polinomio que mejor aproxima a f en sus nodos. Cuanto más se parezca f a un polinomio de bajo grado (suave), más rápido converge.

Convergencia: error vs. número de evaluaciones de f



- Rectángulo y trapecio: rectas de pendiente -1 en esta gráfica, porque $N \propto h^{-2}$ y el error $\propto h^2 = N^{-1}$.
- Gauss: convergencia exponencial para f suave (la curva cae cada vez más rápido).
- Para error $\sim 10^{-10}$: Gauss usa 36 evaluaciones; el trapecio necesitaría del orden de 10^9.

Dominios no rectangulares

Límites variables: $c(x) \le y \le d(x)$

$$I = \int_a^b \int_{c(x)}^{d(x)} f(x,y)\, dy\, dx$$

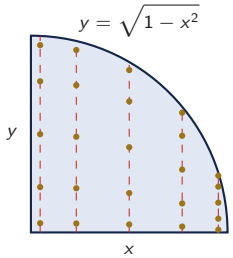
Algoritmo (Gauss en ambas direcciones):

1. Elegir los n nodos x_i de Gauss en $[a, b]$.
2. Para cada x_i, mapear $[c(x_i), d(x_i)]$ a $[-1, 1]$ y aplicar Gauss en y:

$$g(x_i) \approx \frac{d(x_i) - c(x_i)}{2} \sum_j w_j\, f(x_i, y_{ij})$$

3. Integrar g en x: $I \approx \dfrac{b-a}{2} \sum_i w_i\, g(x_i)$.

Los nodos en y cambian con cada x_i: ya no forman una malla regular.



Gauss 5×5 en un cuarto de círculo

Aplicación: momento polar de inercia de un cuarto de círculo

$$J = \iint_D (x^2 + y^2)\, dA, \qquad D: \text{ cuarto de círculo de radio } R = 1, \qquad J_{\text{exacto}} = \frac{\pi R^4}{8} = 0.392\,699\,08$$

Coordenadas cartesianas, $0 \le y \le \sqrt{1 - x^2}$:

n	eval.	error
2	4	$+1.47 \times 10^{-2}$
4	16	$+1.76 \times 10^{-3}$
8	64	$+2.43 \times 10^{-4}$
16	256	$+3.26 \times 10^{-5}$

Coordenadas polares, $dA = r\, dr\, d\theta$:

$$J = \int_0^{\pi/2} \int_0^1 r^3\, dr\, d\theta$$

Integrando polinomial de grado 3 en r y constante en θ: Gauss con 2 nodos en r da el valor exacto.

Idea clave. En cartesianas Gauss converge lento: $d(x) = \sqrt{1 - x^2}$ tiene derivada infinita en $x = 1$ y no se parece a un polinomio. Elegir bien las coordenadas puede valer más que refinar la malla.

Integrales triples y múltiples

Tres dimensiones: la misma receta

En un paralelepípedo $[a_1, b_1] \times [a_2, b_2] \times [a_3, b_3]$:

$$I \approx |J| \sum_i \sum_j \sum_k w_i w_j w_k \, f(x_i, y_j, z_k), \quad |J| = \prod_{m=1}^{3} \frac{b_m - a_m}{2}$$

Trapecio 3D: pesos $c_i c_j c_k \in \{1, 2, 4, 8\}$ (vértices, aristas, caras, interior) y factor $h_x h_y h_z / 8$.

Ejemplo: $\displaystyle\iiint_{[0,1]^3} e^{-(x^2+y^2+z^2)} dV = 0.416\,538\,386$

Método	eval.	error
Gauss 2^3	8	-3.84×10^{-4}
Gauss 3^3	27	-1.60×10^{-5}
Gauss 4^3	64	$+5.61 \times 10^{-7}$
Trapecio $n = 4$	125	-6.39×10^{-3}
Trapecio $n = 8$	729	-1.60×10^{-3}
Trapecio $n = 16$	4913	-4.01×10^{-4}

Gauss con 8 evaluaciones $\approx$ trapecio con 4913.

La maldición de la dimensionalidad

Con n nodos por eje, una regla producto en d dimensiones usa $N = n^d$ evaluaciones. Con $n = 10$:

d	$N = 10^d$
1	10
2	100
3	1 000
6	1 000 000
10	10^{10}

- Para una regla de orden p: error $\propto h^p \propto N^{-p/d}$. Con d grande, el orden "se diluye".
- **Monte Carlo**: error $\propto N^{-1/2}$ independiente de d. Gana a las reglas producto cuando $d \gtrsim 5$ a 10 (finanzas, física estadística, incertidumbre).
- Alternativas intermedias: rejillas dispersas (Smolyak), cuadraturas adaptativas.

Idea clave. Para 2 y 3 dimensiones (la mayoría de problemas de ingeniería mecánica) las reglas producto de Gauss son la herramienta de elección: es exactamente lo que usan los programas de elementos finitos para integrar en cada elemento.

Implementación en Python

Rectángulo y trapecio en 2D (vectorizado con NumPy)

```python
import numpy as np

def rectangulo_2d(f, a, b, c, d, nx, ny):
    hx, hy = (b-a)/nx, (d-c)/ny
    xm = a + hx*(np.arange(nx) + 0.5)
    ym = c + hy*(np.arange(ny) + 0.5)
    X, Y = np.meshgrid(xm, ym, indexing="ij")
    return hx*hy*np.sum(f(X, Y))
```

```python
def trapecio_2d(f, a, b, c, d, nx, ny):
    hx, hy = (b-a)/nx, (d-c)/ny
    x = np.linspace(a, b, nx+1)
    y = np.linspace(c, d, ny+1)
    wx = np.full(nx+1, 2.0); wx[[0,-1]] = 1
    wy = np.full(ny+1, 2.0); wy[[0,-1]] = 1
    W = np.outer(wx, wy)      # pesos 1-2-4
    X, Y = np.meshgrid(x, y, indexing="ij")
    return hx*hy/4*np.sum(W*f(X, Y))
```

```python
f = lambda x, y: np.exp(-(x**2 + y**2))
print(rectangulo_2d(f, 0, 1, 0, 1, 16, 16))   # 0.55792521
print(trapecio_2d(f, 0, 1, 0, 1, 16, 16))     # 0.55738856
```

Idea clave. `np.outer` construye la plantilla de pesos como producto tensorial, igual que en la teoría.

Gauss–Legendre 2D y comparación con SciPy

```python
def gauss_2d(f, a, b, c, d, n):
    t, w = np.polynomial.legendre.leggauss(n)
    x = (b-a)/2*t + (a+b)/2      # [-1,1] -> [a,b]
    y = (d-c)/2*t + (c+d)/2
    X, Y = np.meshgrid(x, y, indexing="ij")
    jac = (b-a)*(d-c)/4          # no olvidarlo!
    return jac*np.sum(np.outer(w, w)*f(X, Y))

for n in range(1, 7):
    print(n, gauss_2d(f, 0, 1, 0, 1, n))
```

```python
from scipy import integrate

# dblquad integra primero en y:
# el orden de argumentos es f(y, x)
I, err = integrate.dblquad(
    lambda y, x: f(x, y),
    0, 1,                          # x
    0, lambda x: np.sqrt(1-x**2) # y
)
```

`dblquad` es *adaptativa*: refina donde hace falta y entrega una estimación del error. Úsela como referencia, no como sustituto de entender el método.

El script completo (`integracion_multiple.py`) incluye la versión con límites variables, la versión 3D y todas las tablas de esta clase.

Síntesis

Resumen comparativo

Método	Nodos	Orden	Exacta para	Cuándo usarla
Rectángulo (punto medio)	centros de celda	$O(h^2)$	bilineales	f singular en el borde
Trapecio	vértices de la malla	$O(h^2)$	bilineales	datos tabulados en una malla
$(2M + T)/3$	ambos	$O(h^4)$	grado total ≤ 3	mejorar M y T sin costo
Gauss $n \times n$	raíces de P_n	exponencial*	grado $2n-1$ por variable	f suave y evaluable donde se quiera
Monte Carlo	aleatorios	$N^{-1/2}$	–	dimensión alta

*para funciones analíticas (muy suaves) en el dominio.

Errores frecuentes. (1) olvidar el jacobiano del cambio de variable; (2) invertir el orden de los argumentos en `dblquad`; (3) usar Gauss sobre una función con discontinuidades o derivadas infinitas sin dividir el dominio o cambiar de coordenadas; (4) comparar métodos por n en vez de por número de evaluaciones.

Ejercicios propuestos

1. **Placa con densidad variable.** Una placa $[0, 0.4] \times [0, 0.3]$ m tiene $\rho(x, y) = 7800\,(1 + 0.5\,e^{-10x}\cos(5y))$ kg/m^3 y espesor 5 mm. Calcule su masa y su centro de masa con trapecio ($n = 4, 8, 16$) y Gauss (2×2, 3×3). ¿Cuántas evaluaciones necesita cada método para 4 cifras correctas?

2. **Compuerta hidrostática.** Calcule la fuerza del agua sobre una compuerta semicircular vertical de radio 1 m cuyo diámetro está en la superficie libre ($p = \rho g y$). Hágalo en cartesianas y en polares, y explique la diferencia de convergencia.

3. **Orden empírico.** Para el ejemplo de referencia, grafique en log–log el error del rectángulo y del trapecio contra h y estime la pendiente. Repita con $(2M + T)/3$.

4. **Integral triple.** Calcule la masa de un bloque $[0, 1]^3$ con densidad $\rho = 1 + xyz + \sin(\pi z)$ con Gauss n^3 y con `tplquad`. ¿Con qué n es exacta la parte polinómica?

5. **(Reto) Monte Carlo.** Estime el volumen de la esfera unitaria en $d = 2, 3, 5, 8$ por Monte Carlo y por una regla producto con el mismo N. ¿A partir de qué d gana Monte Carlo?

Referencias

- Chapra, S. C., Canale, R. P. *Métodos numéricos para ingenieros*, 7.ª ed. McGraw-Hill, 2015. (Integración numérica y cuadratura de Gauss.)
- Burden, R. L., Faires, J. D., Burden, A. M. *Análisis numérico*, 10.ª ed. Cengage, 2017. (Integrales múltiples.)
- Davis, P. J., Rabinowitz, P. *Methods of Numerical Integration*, 2.ª ed. Academic Press, 1984.
- Stroud, A. H. *Approximate Calculation of Multiple Integrals*. Prentice-Hall, 1971.
- Trefethen, L. N. Is Gauss quadrature better than Clenshaw–Curtis? *SIAM Review* 50 (2008) 67–87.
- Documentación: `numpy.polynomial.legendre.leggauss`, `scipy.integrate.dblquad`, `scipy.integrate.tplquad`.